



UTNMDP
Regional Mar del Plata

Base de Datos II

Tema: Cursores

Integrantes:

- Gabriel Ramírez
- Joaquín Nieva
- Iván Salvetti

Comisión 1

Grupo 4

Profesor: Adrián Aroca



Índice

Introducción	3
Cursores: ¿Qué son?, ¿Cómo funcionan?	4-5
Ventajas y desventajas	6
¿Cuándo utilizarlos?	7
Características de los cursores	8-9
Cursores vs. Consultas SQL	10
Cursores vs. Consultas SQL: Tiempos de respuesta	10-11
Resumen conceptual	12
Conclusión	12
Referencias	13

Introducción

En la actualidad, las bases de datos relacionales representan uno de los pilares fundamentales en el almacenamiento, organización y gestión de la información dentro de los sistemas informáticos. Empresas, instituciones y aplicaciones de todo tipo dependen diariamente de bases de datos para administrar grandes volúmenes de información de manera eficiente, segura y estructurada. Dentro de este contexto, el lenguaje SQL se ha consolidado como el estándar más utilizado para interactuar con estos sistemas, permitiendo realizar consultas, modificaciones y distintos procesos sobre los datos almacenados.

A medida que los sistemas evolucionan y las necesidades de procesamiento se vuelven más complejas, surge la necesidad de incorporar herramientas que permitan un mayor control sobre la manipulación de los datos. En este sentido, los cursores SQL adquieren una gran importancia, ya que brindan la posibilidad de recorrer y procesar registros de manera secuencial, facilitando la implementación de operaciones más específicas y personalizadas que no siempre pueden resolverse mediante consultas tradicionales.

El estudio de los cursores resulta relevante dentro del ámbito de la programación de bases de datos debido a que forman parte de las herramientas utilizadas en procedimientos almacenados y programación procedural, permitiendo automatizar tareas, aplicar validaciones complejas y desarrollar soluciones adaptadas a distintos escenarios. Además, comprender su funcionamiento contribuye a un mejor entendimiento de cómo se gestionan internamente los datos y de qué manera pueden optimizarse ciertos procesos dentro de un sistema.

En el presente documento se abordarán los conceptos generales relacionados con los cursores SQL, su finalidad, funcionamiento e importancia dentro de las bases de datos relacionales, proporcionando una visión introductoria sobre su utilidad y aplicación en el desarrollo de soluciones informáticas.



Cursores: ¿Qué son?

Los cursores son objetos SQL que permiten recorrer fila por fila los resultados de una consulta SQL. A diferencia de las operaciones estándar que trabajan sobre el bloque de datos completo, el cursor actúa como puntero para procesar lógicas complejas de manera iterativa, de manera similar a los lenguajes de programación iterativos, como C.

¿Cómo funcionan?

Los cursores constan de un ciclo de vida compuesto por la declaración, la apertura, la recuperación de datos mediante **FETCH**, el control del fin del recorrido mediante un **HANDLER** y el cierre del cursor:

1. **Declaración:** Los cursores se declaran similar a una variable SQL como las usadas dentro de procedimientos almacenados, con el uso de la palabra reservada **DECLARE**, luego se define el nombre del cursor y se coloca la palabra reservada **CURSOR**, una vez hecho esto hay que definir para qué consulta SQL vamos a usar el cursor. Para ello, usamos la palabra reservada **FOR** y, a continuación, colocamos la consulta SQL. Ejemplo:

```
DECLARE cursor_ejemplo CURSOR FOR SELECT id,  
nombre, precio FROM Productos;
```

2. **Apertura:** El siguiente paso es la apertura del cursor y la ejecución de la consulta SQL. Para ello, requerimos dos cosas: la utilización de la palabra reservada **OPEN** y la declaración de un bucle con la palabra reservada **LOOP** y **END LOOP** para definir el alcance del mismo, el bucle funciona de manera similar a un bucle while o for con una variable contadora y su condición de corte la cual definiremos de forma manual con un **IF** y para terminar el bucle usaremos la palabra **LEAVE**. Ejemplo:

```
DECLARE v_final INT DEFAULT 0;  
OPEN cursor_ejemplo;  
bucle:LOOP  
IF v_final = 1 THEN LEAVE bucle;  
END IF;
```



3. **Recuperación:** El siguiente paso es la recuperación de los datos de la tabla. Para ello, previamente se deben haber creado variables locales para poder guardar la información que el cursor procesará de forma iterativa, para realizar la recuperación se utiliza la sentencia ***FETCH ... INTO*** la cual hará que los datos de la fila sobre la que el cursor está iterando se guarden en las variables que declaramos previamente. Ejemplo:

```
FETCH cursor_ejemplo INTO v_id, v_nombre, v_precio;
```

4. **Cierre:** El cierre del cursor consiste en finalizar el bucle y luego cerrar el cursor. Para detectar el final del recorrido, se puede declarar un **HANDLER** de tipo **NOT FOUND**, que modifica una variable de control cuando el cursor ya no encuentra más filas. De esta manera, se cumple la condición de corte, se abandona el bucle y finalmente se cierra el cursor.

Finalmente, el ciclo de vida de un cursor aplicado en SQL se vería de la siguiente manera:

```
DELIMITER //
```

```
CREATE PROCEDURE sp_ejemplo_cursor()  
BEGIN  
  
    DECLARE v_final INT DEFAULT 0;  
    DECLARE v_id INT;  
    DECLARE v_nombre VARCHAR(50);  
    DECLARE v_precio DECIMAL(10,2);  
  
    DECLARE cursor_ejemplo CURSOR FOR SELECT id, nombre, precio FROM Productos;  
  
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_final = 1;  
  
    OPEN cursor_ejemplo;  
  
    bucle : LOOP  
  
        FETCH cursor_ejemplo INTO v_id, v_nombre, v_precio;  
  
        IF v_final = 1 THEN LEAVE bucle;  
        END IF;  
  
        UPDATE Productos SET precio = 125.00 WHERE precio < 100 AND id = v_id;  
  
    END LOOP bucle;  
  
    CLOSE cursor_ejemplo;  
  
END//  
  
DELIMITER ;
```



Ventajas y desventajas

Ventajas

Procesamiento fila por fila: Los cursores permiten trabajar cada registro de manera individual. Esto es útil cuando cada fila necesita un tratamiento diferente, como validar datos, calcular valores o ejecutar una acción específica según el contenido del registro.

Uso de lógica procedural: Permiten utilizar variables, condicionales, bucles y manejadores dentro de un procedimiento almacenado. Gracias a esto, se pueden resolver procesos más complejos que no siempre son fáciles de expresar con una única consulta SQL.

Automatización de tareas: Son útiles para automatizar procesos repetitivos, como generar alertas de stock, registrar historiales, procesar facturación pendiente, migrar datos o crear notificaciones personalizadas para distintos usuarios.

Mayor claridad en procesos complejos: Cuando una consulta SQL se vuelve demasiado extensa o difícil de leer, un cursor puede ayudar a dividir el procedimiento en pasos más claros: obtener datos, recorrerlos, aplicar condiciones y ejecutar acciones.

Desventajas

Menor rendimiento: La principal desventaja es que los cursores procesan los datos fila por fila, mientras que las consultas SQL tradicionales trabajan sobre conjuntos completos. Por eso, en grandes volúmenes de datos suelen ser más lentos.

Mayor consumo de recursos: Al mantener un cursor abierto, MySQL debe administrar el recorrido de los registros, lo que puede consumir más memoria, CPU y tiempo de ejecución.

Código más complejo: Usar cursores requiere una estructura más compleja que una consulta tradicional: declarar variables, declarar el cursor, abrirlo, recorrerlo con FETCH, controlar el fin del recorrido y cerrarlo. Esto aumenta la posibilidad de errores.

Limitaciones en MySQL: En MySQL, los cursores son de solo lectura, no desplazables y asensibles. Esto significa que no permiten modificar directamente la fila actual, no pueden retroceder y el motor decide internamente cómo manejar el conjunto de resultados.



¿Cuándo utilizarlos?

Los cursores en MySQL deben utilizarse cuando se necesita recorrer los registros de una consulta fila por fila y aplicar una lógica específica sobre cada uno de ellos. Son útiles en situaciones en las que una consulta SQL tradicional no resulta suficiente o cuando el proceso requiere decisiones individuales para cada registro.

Una situación común para usar cursores es cuando se deben realizar validaciones o cálculos personalizados por cada fila. Por ejemplo, analizar el saldo de cada cliente, verificar el stock de cada producto o procesar pedidos pendientes según distintas condiciones.

Sin embargo, los cursores deben usarse con cuidado. Si una operación puede resolverse mediante una consulta SQL tradicional con JOIN, GROUP BY, CASE, UPDATE o INSERT INTO ... SELECT, generalmente es preferible evitar el cursor. Esto se debe a que las consultas SQL trabajan sobre conjuntos de datos completos y suelen tener mejor rendimiento.

Ejemplos de dónde utilizarlos:

1) Notificaciones personalizadas:

Ejemplo: Un banco necesita generar una notificación con un mensaje personalizado según la deuda que tenga cada cliente y registrar el aviso.

2) Reglas de negocio especiales:

Ejemplo: Un supermercado necesita aplicar descuentos personalizados, verificar el historial y actualizar la categoría de los clientes para cada pedido realizado.

3) Migración de datos:

Ejemplo: En una base de datos se necesita leer los usuarios viejos, convertir formatos e insertarlos en otra tabla.

ACLARACIÓN: Los cursores en MySQL pueden utilizarse dentro de programas almacenados, como procedimientos almacenados, funciones almacenadas, triggers y eventos.



Características de los cursores

En MySQL, los cursores se utilizan dentro de programas almacenados, especialmente en procedimientos almacenados. A diferencia de otros motores de bases de datos, MySQL no ofrece una gran variedad de tipos de cursores configurables. Sus cursores poseen características específicas: son explícitos, de solo lectura, no desplazables y asensibles.

Esto significa que permiten recorrer el resultado de una consulta **SELECT** fila por fila, pero no permiten retroceder, desplazarse libremente entre registros, modificar directamente la fila actual ni controlar completamente si los cambios realizados en las tablas durante el recorrido se reflejan en el cursor.

Explícitos: En MySQL, los cursores deben declararse manualmente mediante la sentencia **DECLARE ... CURSOR FOR SELECT** Luego deben abrirse con **OPEN**, recorrerse con **FETCH** y cerrarse con **CLOSE**.

No desplazables o de solo avance: El cursor recorre los registros desde la primera fila hasta la última mediante sucesivas instrucciones **FETCH**. No puede retroceder, volver a una fila anterior ni desplazarse libremente entre registros.

De solo lectura: Permiten recorrer el resultado de una consulta **SELECT**, pero no permiten modificar directamente la fila actual mediante el cursor. Si se necesita actualizar información, debe hacerse mediante sentencias independientes como **UPDATE**, **INSERT** o **DELETE**.

Asensibles: MySQL puede decidir internamente si el cursor trabaja directamente sobre los datos originales o sobre una copia temporal del resultado. Por eso, no siempre se garantiza cómo se reflejan los cambios realizados en las tablas mientras el cursor está abierto.



Ejemplo de cursor en MySQL:

```
DELIMITER //
```

```
CREATE PROCEDURE procesar_facturacion_clientes()
BEGIN
    -- 1. Declarar variables para almacenar los datos de cada
    fila
    DECLARE v_cliente_id INT;
    DECLARE v_saldo_actual DECIMAL(10,2);
    DECLARE v_fin BOOLEAN DEFAULT FALSE;

    -- 2. Declarar el cursor
    -- En MySQL, los cursores son de solo lectura y solo avance
    por defecto
    DECLARE cur_clientes CURSOR FOR SELECT id, saldo FROM
    Clientes WHERE saldo > 0;

    -- Handler para detectar cuando ya no quedan más filas
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_fin = TRUE;

    -- 3. Abrir el cursor
    OPEN cur_clientes;

    -- 4. Bucle para recorrer todas las filas devueltas
    recorrer_clientes: LOOP

        -- Leer la siguiente fila
        FETCH cur_clientes INTO v_cliente_id, v_saldo_actual;

        -- Si no hay más filas, salir del bucle
        IF v_fin THEN LEAVE recorrer_clientes;
        END IF;

        -- LÓGICA DE NEGOCIO:
        -- Procesamos la fila actual.
        -- Insertamos un registro en el historial de facturación para
        este cliente.
        INSERT INTO HistorialFacturacion
        (ClienteId,MontoFacturado,FechaProceso)
        VALUES (v_cliente_id,v_saldo_actual,NOW());

    END LOOP;

    -- 5. Cerrar el cursor
    CLOSE cur_clientes;
END//

DELIMITER ;
```



Cursores vs. Consultas SQL

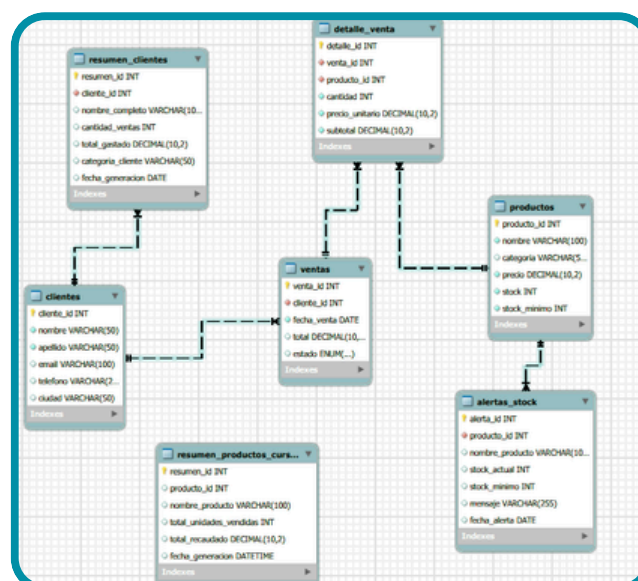
Características	Cursores	Consultas tradicionales
Forma de trabajo	Procesan los datos fila por fila	Procesan conjuntos completos de datos
Rendimiento	Más lento en grandes volúmenes	Más rápido y optimizado
Complejidad de código	Más claro para lógica compleja	Puede volverse muy difícil de leer
Uso de lógica procedural	"Permite IF, variables, loops y validaciones por registro"	"Limitado a expresiones SQL (CASE, subconsultas, etc.)"
Escalabilidad	Consume más recursos	Escala mejor
Mantenimiento	Más mantenible cuando hay muchas reglas de negocio	Las consultas grandes son difíciles de modificar
Automatización de procesos	Muy útil	Limitado
Objetivo principal	Resolver lógica procedural	Manipular datos de forma eficiente

RECORDATORIO: Las consultas SQL son mejores para operaciones masivas y eficientes sobre conjuntos de datos. Los cursores se utilizan cuando la lógica fila por fila es demasiado compleja para resolverse cómodamente con una única consulta SQL.

Tiempos de respuesta

Ahora veamos la diferencia de tiempos de respuesta entre un cursor dentro de un procedimiento almacenado y una consulta SQL tradicional. A partir de esta base de datos y del procedimiento almacenado utilizado, se comparan los tiempos de respuesta de ambos enfoques. A continuación, se muestra la diferencia de tiempos de respuesta:

DER de la base de datos:





Procedimiento almacenado + cursor utilizado:

```
8 DELIMITER //
```

```
9 CREATE PROCEDURE generar_resumen_productos_cursor()
```

```
10 BEGIN
```

```
11     DECLARE v_producto_id INT;
```

```
12     DECLARE v_nombre_producto VARCHAR(100);
```

```
13     DECLARE v_total_unidades INT DEFAULT 0;
```

```
14     DECLARE v_total_recaudado DECIMAL(10,2) DEFAULT 0;
```

```
15     DECLARE v_fin BOOLEAN DEFAULT FALSE;
```

```
16
```

```
17     DECLARE cursor_productos CURSOR FOR SELECT producto_id, nombre FROM productos;
```

```
18
```

```
19     DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_fin = TRUE;
```

```
20     OPEN cursor_productos;
```

```
21
```

```
22     recorrer_productos: LOOP
```

```
23
```

```
24         FETCH cursor_productos INTO v_producto_id, v_nombre_producto;
```

```
25
```

```
26         IF v_fin THEN LEAVE recorrer_productos;
```

```
27     END IF;
```

```
28
```

```
29     SELECT IFNULL(SUM(dv.cantidad), 0), IFNULL(SUM(dv.subtotal), 0)
```

```
30     INTO v_total_unidades, v_total_recaudado
```

```
31     FROM detalle_venta dv INNER JOIN ventas v ON dv.venta_id = v.venta_id
```

```
32     WHERE dv.producto_id = v_producto_id AND v.estado = 'PAGADA';
```

```
33
```

```
34     INSERT INTO resumen_productos_cursor (producto_id,nombre_producto,total_unidades_vendidas,total_recaudado,fecha_generacion)
```

```
35     VALUES (v_producto_id,v_nombre_producto,v_total_unidades,v_total_recaudado,NOW());
```

```
36     END LOOP;
```

```
37     CLOSE cursor_productos;
```

```
38 END//
```

```
39 DELIMITER ;
```

Tiempo de respuesta de procedimiento almacenado con cursor:

Result Grid					
Filter Rows:		Export:		Wrap Cell Content:	
	metodo	inicio	fin	tiempo_microsegundos	tiempo_milisegundos
▶	CON CURSOR	2026-05-24 19:33:56.908245	2026-05-24 19:33:56.977725	69480	69.4800

Tiempo de respuesta de consulta tradicional SQL:

	Query_ID	Duration	Query
	1	0.00121475	SHOW WARNINGS
	2	0.00111275	SET profiling = 1
	3	0.00874250	SHOW WARNINGS
▶	4	0.00317875	EXPLAIN SELECT p.producto_id, p.nombre AS nombre_p...



Resumen conceptual

Concepto	¿Qué significa?	Ejemplo
DECLARE	Se utiliza para declarar variables, cursores o handlers dentro de un procedimiento almacenado.	<code>DECLARE v_final INT DEFAULT 0;</code>
DECLARE CURSOR	Define el cursor y la consulta <code>SELECT</code> que será recorrida fila por fila.	<code>DECLARE cur_clientes CURSOR FOR SELECT id, saldo FROM clientes;</code>
OPEN	Abre el cursor y ejecuta la consulta asociada para preparar el recorrido de los registros.	<code>OPEN cur_clientes;</code>
FETCH	Recupera la siguiente fila del cursor y guarda sus valores en variables locales.	<code>FETCH cur_clientes INTO v_id, v_saldo;</code>
HANDLER	Controla situaciones especiales, como detectar cuándo el cursor ya no tiene más filas.	<code>DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_final = 1;</code>
LOOP	Permite repetir instrucciones mientras se recorren las filas obtenidas por el cursor.	<code>bucle: LOOP ... END LOOP;</code>
CLOSE	Cierra el cursor y libera los recursos utilizados durante el recorrido.	<code>CLOSE cur_clientes;</code>
Forward-only	Significa que el cursor solo puede avanzar desde la primera fila hasta la última. No puede retroceder.	<code>FETCH</code> avanza fila por fila en una sola dirección.
Read-only	Significa que el cursor solo permite leer los datos del resultado de una consulta. No modifica directamente la fila actual.	<code>SELECT id, saldo FROM clientes;</code>

Conclusión

En conclusión, los cursores son una herramienta útil dentro de las bases de datos cuando se necesita realizar un procesamiento detallado y secuencial de los datos, permitiendo aplicar lógica compleja, validaciones y automatizaciones fila por fila.

Sin embargo, debido a su menor rendimiento y mayor consumo de recursos, deben utilizarse únicamente en situaciones en las que una consulta SQL tradicional no sea suficiente o cuando esta se vuelva demasiado compleja y difícil de mantener.

Por lo tanto, es importante elegir correctamente entre cursores y consultas SQL según las necesidades y la complejidad del proceso a desarrollar, ya que una mala decisión puede afectar el rendimiento de la base de datos y provocar un uso ineficiente de los recursos.



Referencias

Oracle. (s. f.). MySQL 8.4 Reference Manual: Cursors. MySQL. Recuperado el 24 de mayo de 2026, de <https://dev.mysql.com/doc/refman/8.4/en/cursors.html>

Oracle. (s. f.). MySQL 8.4 Reference Manual: Cursor DECLARE statement. MySQL. Recuperado el 24 de mayo de 2026, de <https://dev.mysql.com/doc/refman/8.4/en/declare-cursor.html>

Oracle. (s. f.). MySQL 8.4 Reference Manual: Cursor OPEN statement. MySQL. Recuperado el 24 de mayo de 2026, de <https://dev.mysql.com/doc/refman/8.4/en/open.html>

Oracle. (s. f.). MySQL 8.0 Reference Manual: EXPLAIN statement. MySQL. Recuperado el 24 de mayo de 2026, de <https://dev.mysql.com/doc/refman/8.0/en/explain.html>

Oracle. (s. f.). MySQL 8.4 Reference Manual: Cursor FETCH statement. MySQL. Recuperado el 24 de mayo de 2026, de <https://dev.mysql.com/doc/refman/8.4/en/fetch.html>

Oracle. (s. f.). MySQL 8.4 Reference Manual: Cursor CLOSE statement. MySQL. Recuperado el 24 de mayo de 2026, de <https://dev.mysql.com/doc/refman/8.4/en/close.html>

Oracle. (s. f.). MySQL 8.4 Reference Manual: DECLARE HANDLER statement. MySQL. Recuperado el 24 de mayo de 2026, de <https://dev.mysql.com/doc/refman/8.4/en/declare-handler.html>

OpenAI. (2026). ChatGPT (GPT-5.5 Thinking) [Modelo de lenguaje grande]. <https://chat.openai.com/>